Agentic AI for Economic Research

Session 2: live workflows, case studies, and lessons learned

Marco Le Moglie
Catholic University of the Sacred Heart

Agentic Economic Research seminar | Session 2

Mini-course for PhD students and faculty | hours 3–4

Slides source on GitHub

What we see today

1. Agent interfaces live screens

Chat, desktop apps, IDE, CLI — menus and options in real tools.

2. Success stories projects at equilibrium

Selected projects where the agent workflow is clean and repeatable.

3. Second brain Obsidian vault & memory

How it works; how I organise mine; how you might organise yours.

4. Work in progress live screens

Bootstrap, supervision, errors — all on real screens.

From vocabulary to live practice — always with supervision.

Part 1 – Agent interfaces

Part 1: four interface families, live

Today we open the real tools

- **Chat window** — e.g. ChatGPT-style UI: model picker, attachments, memory toggles, project folders.
- **Desktop agent apps** — Codex, Claude, Cursor, Antigravity: same brain, different shell.
- **IDE agents** — editor-embedded: rules, MCP, diff view, terminal panel.
- **CLI in the terminal** — scriptable, composable, closest to reproducibility.

Live demo goal

Not a feature tour for its own sake: show *where* autonomy lives in each interface — and which menu choices matter for research (privacy, context, tools, logs).

Session 1 gave the map; Session 2 opens the cockpit.

Part 2 – Success stories

Part 2: projects at equilibrium

For each case: objective, current workflow, what we learned, problems hit.

- **YouTube channel** — economics explainer pipeline.
- **Newsletter** — ASSIST Weekly harvest-to-send stack.
- **Thesis evaluation** — skill for grading Catholic University theses.
- **Mafia culture** — datasets, reports, documentation agents.
- **Paper review** — referee-report skills and adversarial checks.
- **Fragmented opposition** — new sources & dataset construction.
- **Purge** — journal-target skill adapted to a new outlet.

Common thread: repeatable folder discipline + human gates.

Success story: *Cosa fanno gli Economisti*

Live links: YouTube | Instagram | Facebook

Objective

- Turn economics papers into short explainer videos for a general audience.
- Automate production, SEO, YouTube programming, and cross-posting (FB/IG).

Current state & workflow

- End-to-end: paper → cover → NotebookLM video → cleanup → multilingual upload → Buffer social.
- **Codex chat** = control plane; runner `./workflow` + scripts in `Execution/`.
- Three layers: *Directives* (SOP) → orchestration → deterministic execution.
- Lanes: Enea (production), Romolo (YouTube/SEO), Marcello (social), Mercurio (backup), Ulisse (scouting).

What I learned

- **Connectors**: MCP (NotebookLM), APIs (YouTube, Buffer), CLI (`nlm`, `./workflow`).
- **Structured rules** + **multi-agent** workflows; truth in `video_tracking.json`.
- **Token control**: targeted git push, recover partial uploads, no blind re-runs.
- **YouTube SEO**: titles, playlists, competitor comments, monthly report.
- **Tool drift**: Antigravity/Codex upgrades broke rules, MCP paths, terminal defaults — re-audit SOPs after each version jump.

Problems hit

- Buffer 404 until assets on GitHub; NotebookLM MCP vs. CLI cookie mismatch.
- YouTube thumbnail >2 MB after upload; Telegram retired → Codex-first.

Success story: *ASSIST Weekly* (newsletter)

Live links: assistweekly.org | Issue #13 (web)

Objective

- Weekly brief: datasets, tools/methods, working papers.
- Stack: discover ⟶ QA ⟶ email ⟶ landing/signup.

Current state

- #13 sent 2026-07-06; `2026-07-13` shortlist ready.
- Brevo campaigns; Cloudflare Pages/Worker; internal dashboard.

Three automations

- **Wed** harvest ⟶ shortlist + dashboard
- **Thu** render after `APPROVED.flag`
- **Mon** Brevo send after `EMAIL_APPROVED`

Manual gates: dashboard review ⟶ flags.

What I learned

- Brevo; Cloudflare deploy; editorial dashboard.
- Scraping/APIs (GitHub, Dataverse, NBER, CEPR).
- Brand/trademark; anti-slop frontend; LaunchAgents.

Problems hit

- Source selection & over-harvest filters.
- Manual ratings train judgment between automations.

Success story: *Mafia Culture* (Neomelodic Corpus)

Repo: github.com/MarcoLeMoglie/Mafia_culture

Objective

- Build a reproducible corpus of Italian neomelodic music (artists, tracks, lyrics) for cultural economics.

Current state

- **190** artists; **32.5k** title-level tracks; **2.6k** high-confidence lyrics in reports.
- Four LaTeX PDF reports (links →); canonical layers in `analysis/`.

Pilot corpus steps (legacy pipeline)

- Seed artists → metadata harvest → dedup export → Stata layers → standardized PDF reports.

Spotify frontier (vs. core artist seeds)

- `seed-core` (190) → `discover-network` → `sync-mirror` → LRCLib lyrics → Jaccard export.
- Goal: full discographies + audio features beyond the original artist list.

What I learned

- **Machine generates, human verifies**: sample construction + exploratory QA loops.
- **Standardized LaTeX reports** as control documents; branch + commit at each dataset decision.
- **Cold-start replication package** when a layer is “ready”.

Tracks report — malavita word cloud

Report PDFs

Corpus | Artists | Tracks | Lyrics

Problems hit

- Layer drift (raw vs. canonical vs. compatibility surfaces).
- Spotify API limits, album dedup, partial lyrics coverage (∼20–40%).

Success story: paper peer review — skill `/referaggio`

Objective: turn an annotated manuscript into a constructive English referee report in Marco's voice, with adversarial checks and style archive.

Skill `/referaggio` — main steps

1. **Intake** — Marco uploads annotated PDF to `reviews/`.
2. **Scaffold** — folder `{MS-ID}-{JOURNAL}/` + checklist.
3. **Comments** — extract, expand, organize; drop nonsense *with reason*.
4. **Econoclast** — adversarial pass; manual PDF verify each finding.
5. **Plan** — `proposed_points.md`; STOP: Marco approves.
6. **Draft** — `referee_report.tex/md/pdf` (conciliatory tone).
7. **Anti-slop** — `prose-check` on draft.
8. **Archive** — style example; STOP: final Marco approval.

Human gates (non-negotiable)
Wait for PDF upload (step 1). Clarify ambiguous comments (step 3). Approve integrated plan before drafting (step 5). Approve final report before archive (step 8). Econoclast supplements — does not replace Marco's judgment.

Stack: Brain `referee-reports/`, Dropbox `reviews/`, Econoclast, `tools/prose-check`, Style examples.

Success story: *Purge* — skill `journal-style-paper-rewrite`

Objective: adapt a frozen paper (e.g. JOP → Demography) — new framing and prose, *same* empirical package.

Skill workflow — main parts

1. **Lock source** — read memory; do not edit frozen paper.
2. **Preserve empirics** — no new regressions/tables unless Marco reopens.
3. **Design prompt** — ask Marco if missing (journal, framing, tone).
4. **Target-journal plan** — title/abstract, intro, results rhetoric.
5. **Anti-slop** — prose-check on new version.
6. **Two outputs** — `AA_paper_<journal>.tex` + matching report.
7. **Color logic** — mark only text actually changed vs. source.
8. **Compile & verify** — source untouched; report matches new version.

Human gates

Frozen canonical paper stays read-only. Journal-design prompt required before any rewrite. Marco approves framing; agents do not improvise empirical reopening. Color only real edits — not decoration.

Example: `AA_paper_JOP.tex` frozen → `AA_paper_Demography_full.tex` + `AA_report_Demography_full.tex`.

Part 3 – Second brain

Part 3: memory and the Obsidian second brain

- Session 1 introduced *why* a curated Markdown vault beats blind RAG for research traceability.
- Today: **how** it works in practice — my vault layout, project folders, session start/end prompts, and what *not* to store.

We will show

- Areas, projects, wiki vs. raw notes.
- Brain bootstrap: INDEX, CURRENT_STATE, NEXT_ACTIONS.
- Memanto as semantic layer *on top of* Markdown truth.
- Skills and integrations folder.

Takeaway for you

- Start small: one project folder, eight canonical files.
- Write decisions in Markdown, not only in chat.
- Treat the vault as the restart point every session.

Part 4 – Work in progress

Part 4: live WIP — setup, supervision, and fixes

1. Project bootstrap (live)

- My standard start: **Brain** (INDEX, CURRENT_STATE, wiki) → **GitHub** repo → **Overleaf** sync.
- **Example:** *Mafia migration* — matched DiD pipeline, analysis plan, wiki, push to GitHub.

2. When the machine needs a human gate

- **Purge** — 1940 INE profession proxies: terrible embedded OCR; rebuilt with image re-OCR, column/row templates, census caps; I spot-checked flagged cells.
- Pattern: agent proposes fix → I verify against source → we lock the auditable pipeline.

3. Interactive contrast

- Build the same table with **plain ChatGPT** vs. a **Codex coding agent** on real project files.

Live demos may fail — that is part of the lesson.